

An Analysis of Sparse attention transformer

Sparse attention transformer

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Sparse attention transformer -- Part 1

Like earlier seq2seq models, the original transformer model used an encoder-decoder architecture. The encoder consists of encoding layers that process all the input tokens together one layer after another, while the decoder consists of decoding layers that iteratively process the encoder's output and the decoder's output tokens so far. The purpose of each encoder layer is to create contextualized representations of the tokens, where each representation corresponds to a token that "mixes" information from other input tokens via self-attention mechanism. Each decoder layer contains two attention sublayers: (1) cross-attention for incorporating the output of encoder (contextualized input token representations), and (2) self-attention for "mixing" information among the input tokens to the decoder (i.e. the tokens generated so far during inference time). Both the encoder and decoder layers have a feed-forward neural network for additional processing of their outputs and contain residual connections and layer normalization steps. These feed-forward layers contain most of the parameters in a transformer model.

Sparse attention transformer -- Part 2

```
/* first sublayer */ z_d_copy ← copy(z_d) for each t in 1:length(z_d) do z_d[t] ← layer.layer_norm(z_d[t]) z_d ← layer.masked_multihead_attention(z_d, z_d, z_d) for each t in 1:length(z_d) do z_d[t] ← z_d[t] + z_d_copy[t]
```

Sparse attention transformer -- Part 3

Relative Position Encodings Relative Position Encodings is similar to ALiBi, but more generic: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QKT}{dk+B}\right)V$ where B is a Toeplitz matrix, that is, $B_{i,j} = B_{i',j'}$ whenever $i-j = i'-j'$. This is contrasted with the original sinusoidal positional encoding, which is an "absolute positional encoding".

Sparse attention transformer -- Part 4

Efficient implementation The transformer model has been implemented in standard deep learning frameworks such as

TensorFlow and PyTorch. Transformers is a library produced by Hugging Face that supplies transformer-based architectures and pretrained models.

Sparse attention transformer -- Part 5

RoPE RoPE (rotary positional embedding), is best explained by considering a list of 2-dimensional vectors $[(x_1(1), x_1(2)), (x_2(1), x_2(2)), (x_3(1), x_3(2)), \dots]$. Now pick some angle θ . Then RoPE encoding is $\text{RoPE}(x_m(1), x_m(2), m) = (\cos m\theta - \sin m\theta \sin m\theta \cos m\theta, (\cos m\theta + \sin m\theta \sin m\theta \cos m\theta))$. Equivalently, if we write the 2-dimensional vectors as complex numbers $z_m := x_m(1) + ix_m(2)$, then RoPE encoding is just multiplication by an angle: $\text{RoPE}(z_m, m) = e^{im\theta} z_m$. For a list of $2n$ -dimensional vectors, a RoPE encoder is defined by a sequence of angles $(\theta(1), \dots, \theta(n))$. Then the RoPE encoding is applied to each pair of coordinates. The benefit of RoPE is that the dot-product between two vectors depends on their relative location only: $\text{RoPE}(x, m)^T \text{RoPE}(y, n) = \text{RoPE}(x, m+k)^T \text{RoPE}(y, n+k)$.

Sparse attention transformer -- Part 6

Alternative attention graphs The standard attention graph is either all-to-all or causal, both of which scales as $O(N^2)$ where N is the number of tokens in a sequence. Reformer (2020) reduces the computational load from $O(N^2)$ to $O(N \ln N)$ by using locality-sensitive hashing and reversible layers. Sparse attention uses attention graphs that grows slower than $O(N^2)$. For example, BigBird (2020) uses random small-world networks which grows as $O(N)$. Ordinary transformers require a memory size that is

An Analysis of Sparse attention transformer

Sparse attention transformer

quadratic in the size of the context window. Attention-free transformers reduce this to a linear dependence while still retaining the advantages of a transformer by linking the key to the value.

Sparse attention transformer -- Part 7

An encoder consists of an embedding layer, followed by multiple encoder layers. Each encoder layer consists of two major components: a self-attention mechanism and a feed-forward layer. It takes an input as a sequence of input vectors, applies the self-attention mechanism, to produce an intermediate sequence of vectors, then applies the feed-forward layer for each vector individually. Schematically, we have: given input vectors $h_0, h_1, \dots$ combine them into a matrix $H = [h_0 \ h_1 \ \dots]$ EncoderLayer $(H) = [FFN(\text{MultiheadAttention}(H, H, H)_0) \text{FFN}(\text{MultiheadAttention}(H, H, H)_1) \ \dots]$

$$\begin{aligned} & \text{given input vectors} \\ & h_0, h_1, \dots \quad \text{combine them into a matrix} \\ & H = \begin{bmatrix} h_0 \\ h_1 \\ \vdots \end{bmatrix} \\ & \text{EncoderLayer}(H) = \begin{bmatrix} \text{FFN}(\text{MultiheadAttention}(H, H, H)_0) \\ \text{FFN}(\text{MultiheadAttention}(H, H, H)_1) \\ \vdots \end{bmatrix} \end{aligned}$$

Sparse attention transformer -- Part 8

The original version of the transformer architecture was proposed in the 2017 paper "Attention Is All You Need" by researchers at Google. The predecessors of transformers were developed as an improvement over previous architectures for machine translation, but have found many applications since. They are used in large-scale natural language processing, computer vision (vision transformers), reinforcement learning, audio, multimodal learning, robotics, playing chess and at disaster responses. It has also led to the development of pre-trained systems, such as generative pre-trained transformers (GPTs) and BERT (bidirectional encoder representations from transformers).